```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
image_path = "/content/Panda.png"

img = cv2.imread(image_path)

if img is None:
    print("Image not found. Please check the image path.")
else:
    print("Image loaded successfully!")
```

```
Image loaded successfully!
```

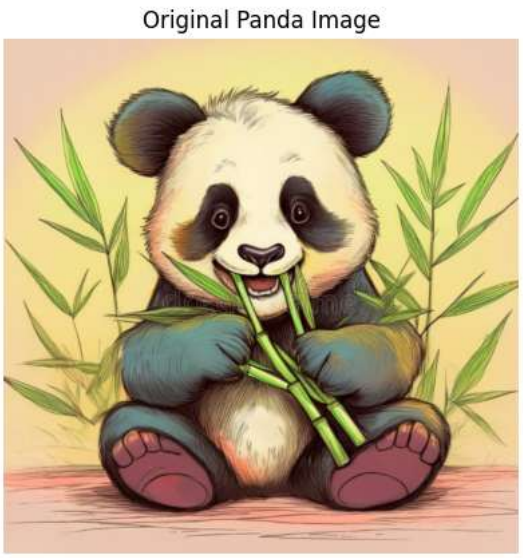
```
cv2_imshow = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.imshow(cv2_imshow)
plt.axis("off")
plt.title("Original Image")
plt.show()
```




```
rgb_img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(6, 5))
plt.imshow(rgb_img)
plt.title("Original Panda Image")
plt.axis("off")
plt.show()
```




```
height, width, channels = img.shape

print("Image Dimensions:")
print("Height:", height, "pixels")
print("Width:", width, "pixels")
print("Number of Channels:", channels)
print("Data Type:", img.dtype)
print("Resolution:", width, "x", height)
```

```
Image Dimensions:
Height: 707 pixels
Width: 707 pixels
Number of Channels: 3
Data Type: uint8
Resolution: 707 x 707
```

```
jpeg_path = "/content/panda_saved.jpg"
png_path = "/content/panda_saved.png"

cv2.imwrite(jpeg_path, img)
cv2.imwrite(png_path, img)

print("Image saved successfully!")
print("JPEG:", jpeg_path)
print("PNG:", png_path)
```

```
Image saved successfully!
JPEG: /content/panda_saved.jpg
PNG: /content/panda_saved.png
```

```
jpeg_img = cv2.imread(jpeg_path)
png_img = cv2.imread(png_path)

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(jpeg_img, cv2.COLOR_BGR2RGB))
plt.title("JPEG Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(png_img, cv2.COLOR_BGR2RGB))
plt.title("PNG Image")
plt.axis("off")

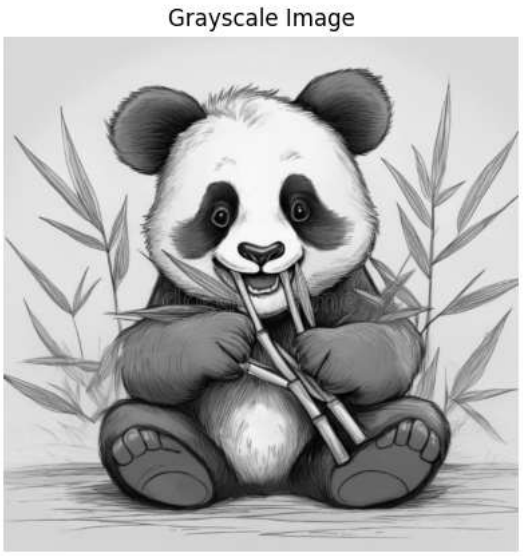
plt.show()
```





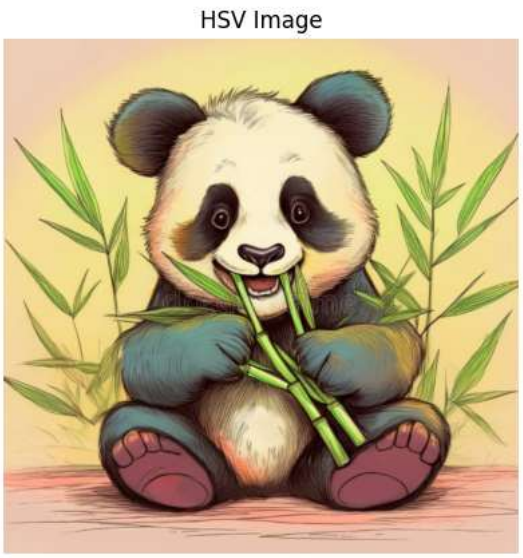

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

plt.figure(figsize=(6, 5))
plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")
plt.show()
```




```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

plt.figure(figsize=(6, 5))
plt.imshow(cv2.cvtColor(hsv, cv2.COLOR_HSV2RGB))
plt.title("HSV Image")
plt.axis("off")
plt.show()
```



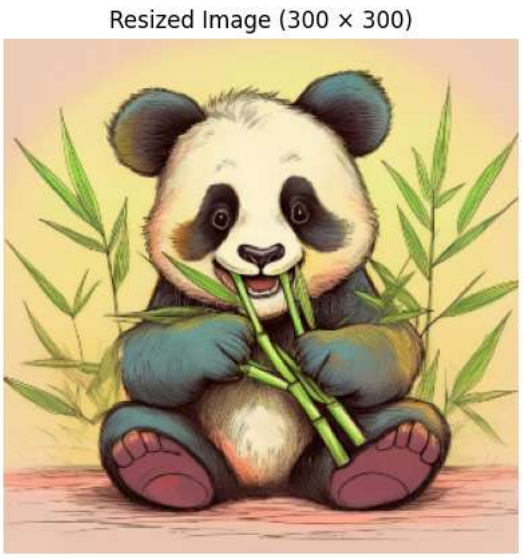

```
lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)

plt.figure(figsize=(6, 5))
plt.imshow(cv2.cvtColor(lab, cv2.COLOR_LAB2RGB))
plt.title("LAB Image")
plt.axis("off")
plt.show()
```




```
resized = cv2.resize(img, (300, 300))

plt.figure(figsize=(6, 5))
plt.imshow(cv2.cvtColor(resized, cv2.COLOR_BGR2RGB))
plt.title("Resized Image (300 × 300)")
plt.axis("off")
plt.show()
```




```
height, width = img.shape[:2]

center = (width // 2, height // 2)

rotation_matrix = cv2.getRotationMatrix2D(center, 45, 1.0)

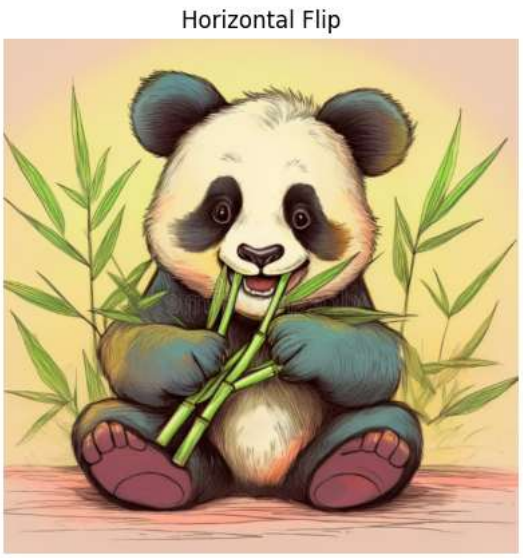
rotated = cv2.warpAffine(img, rotation_matrix, (width, height))

plt.figure(figsize=(6, 5))
plt.imshow(cv2.cvtColor(rotated, cv2.COLOR_BGR2RGB))
plt.title("Rotated Image (45°)")
plt.axis("off")
plt.show()
```



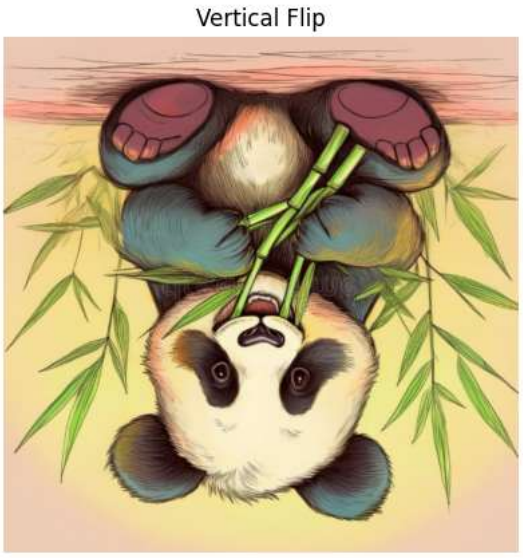

```
horizontal_flip = cv2.flip(img, 1)

plt.figure(figsize=(6, 5))
plt.imshow(cv2.cvtColor(horizontal_flip, cv2.COLOR_BGR2RGB))
plt.title("Horizontal Flip")
plt.axis("off")
plt.show()
```




```
vertical_flip = cv2.flip(img, 0)

plt.figure(figsize=(6, 5))
plt.imshow(cv2.cvtColor(vertical_flip, cv2.COLOR_BGR2RGB))
plt.title("Vertical Flip")
plt.axis("off")
plt.show()
```




```
negative = 255 - img

plt.figure(figsize=(6, 5))
plt.imshow(cv2.cvtColor(negative, cv2.COLOR_BGR2RGB))
plt.title("Negative / Complement Image")
plt.axis("off")
plt.show()
```

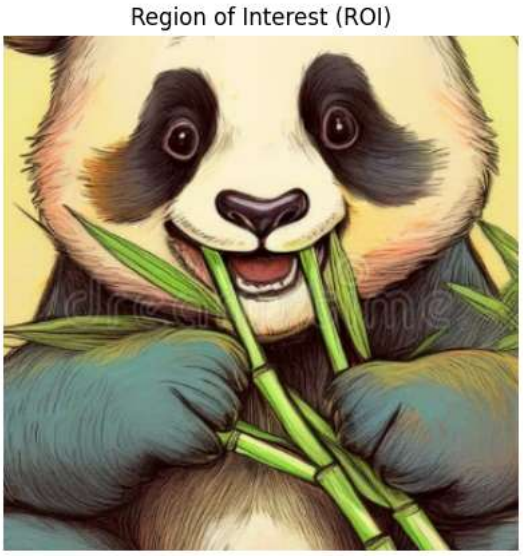



```
height, width = img.shape[:2]

x1 = width // 4
x2 = 3 * width // 4
y1 = height // 4
y2 = 3 * height // 4

roi = img[y1:y2, x1:x2]

plt.figure(figsize=(6, 5))
plt.imshow(cv2.cvtColor(roi, cv2.COLOR_BGR2RGB))
plt.title("Region of Interest (ROI)")
plt.axis("off")
plt.show()
```




```
roi_height, roi_width, roi_channels = roi.shape

print("ROI Properties:")
print("Height:", roi_height)
print("Width:", roi_width)
print("Channels:", roi_channels)
print("Data Type:", roi.dtype)
```

```
ROI Properties:
Height: 354
Width: 354
Channels: 3
Data Type: uint8
```

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

```
plt.figure(figsize=(15, 10))

plt.subplot(2, 3, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

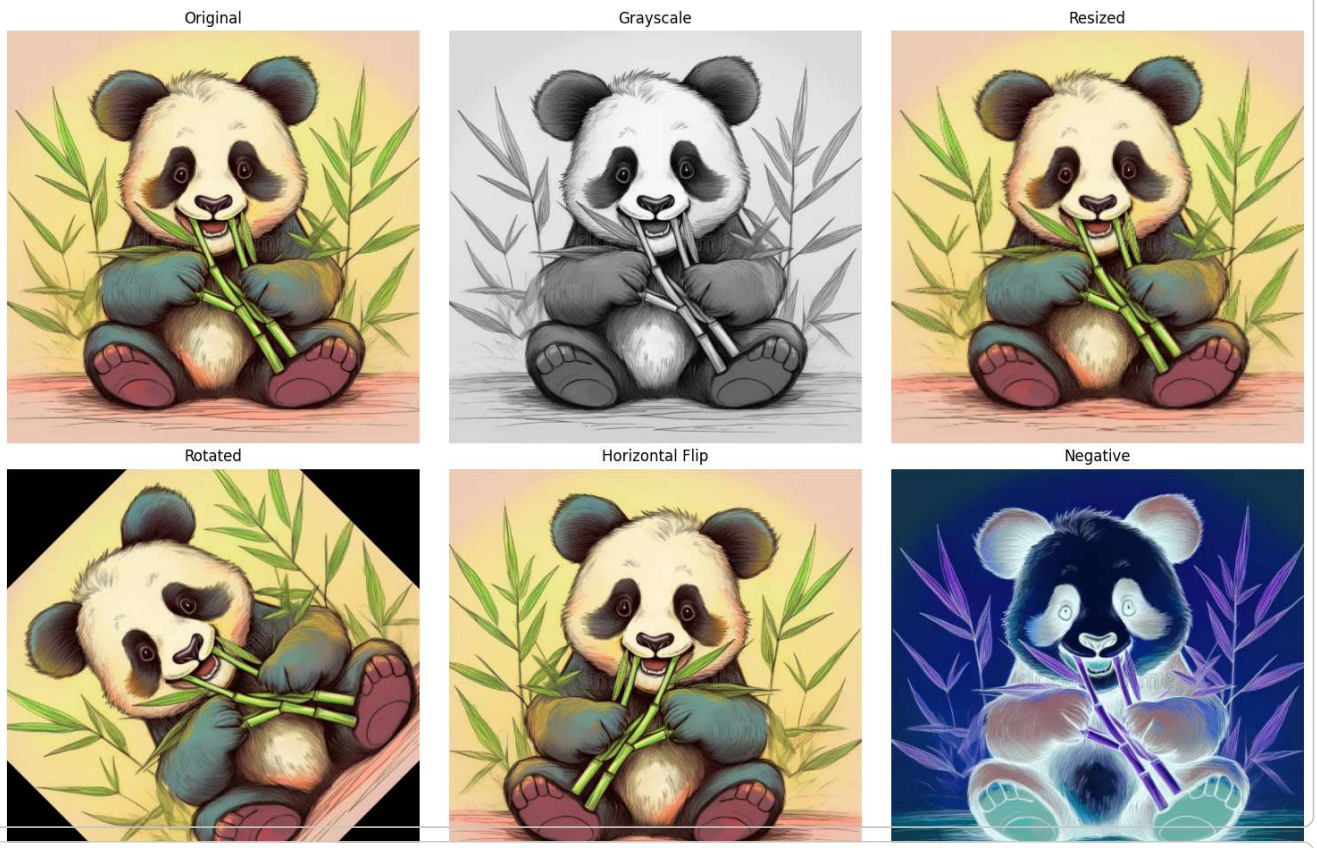
plt.subplot(2, 3, 3)
plt.imshow(cv2.cvtColor(resized, cv2.COLOR_BGR2RGB))
plt.title("Resized")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(cv2.cvtColor(rotated, cv2.COLOR_BGR2RGB))
plt.title("Rotated")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(cv2.cvtColor(horizontal_flip, cv2.COLOR_BGR2RGB))
plt.title("Horizontal Flip")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(cv2.cvtColor(negative, cv2.COLOR_BGR2RGB))
plt.title("Negative")
plt.axis("off")

plt.tight_layout()
plt.show()
```




```
plt.figure(figsize=(15, 5))

plt.subplot(1, 4, 1)
plt.imshow(rgb_img)
plt.title("Original RGB")
plt.axis("off")

plt.subplot(1, 4, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

plt.subplot(1, 4, 3)
plt.imshow(cv2.cvtColor(hsv, cv2.COLOR_HSV2RGB))
plt.title("HSV")
plt.axis("off")

plt.subplot(1, 4, 4)
plt.imshow(cv2.cvtColor(lab, cv2.COLOR_LAB2RGB))
plt.title("LAB")
plt.axis("off")

plt.tight_layout()
plt.show()
```

